

P1868R2

width: clarifying units of width and precision in std::format

Published Proposal, 2020-02-11

This version:

<http://fmt.dev/P1868R1.html>

Authors:

[Victor Zverovich](#)

[Zach Laine](#)

Audience:

LWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

- 1 Introduction**
- 2 Changes**
- 3 SG16 poll results**
- 4 LEWG poll results**
- 5 Motivating example**
- 6 Prior art**
- 7 Locale and execution encodings**
- 8 Windows**
- 9 Precision**
- 10 Proposal**
- 11 Why not code units?**
- 12 Limitations**
- 13 Examples**
- 14 Implementation**
- 15 Wording**
- 16 Acknowledgements**

References

Informative References

“We demand rigidly defined areas of doubt and uncertainty!”

— Douglas Adams

§ 1. Introduction

A new text formatting facility ([P0645]) was adopted into the draft standard for C++20 in Cologne. Unfortunately it left unspecified units of width and precision which created an ambiguity for string arguments in variable-width encodings ([LWG3290]). This paper proposes fixing this shortcoming and specifying width and precision in a way that satisfies the following goals:

- addressing the main use case
- locale-independence by default
- Unicode support
- ordinary and wide execution encoding support
- consistency with the SG16's long-term direction
- following existing practice
- ease of implementation

§ 2. Changes

- Replace "required to display it in a terminal" with "appropriate for displaying it in a terminal".
- Replace

[*Note:* the implementations are encouraged to use Unicode on platforms capable of displaying Unicode text in a terminal which is the case for Windows-based and many POSIX-based operating systems. — end note]

with

Implementations should use Unicode on platforms capable of displaying Unicode text in a terminal. [*Note:* This is the case for Windows-based and many POSIX-based operating systems. — end note]

- Replace "Width of a string in a Unicode encoding is the sum" with "Implementations should estimate the width of a string in a Unicode encoding as the sum".
- Remove the note saying "The method of estimated width computation is subject to change. — end note".
- Add "Unicode® 12.1.0 Standard Annex #29 Unicode Text Segmentation" to Normative references [intro.refs].

- Replace

For string types this field specifies the maximum width. Trailing grapheme clusters or implementation-defined units of width that exceed the given precision are ignored.

with

For string types, this field provides an upper bound for the estimated width of the prefix of the input string that is copied into the output. For a string assumed to be in a Unicode encoding, the longest prefix of whole extended grapheme clusters whose estimated width is no greater than the precision is copied to the output.

- Fix minor wording issues.
- Replace

For the purposes of width computation the string is assumed to be in a fixed operating system dependent encoding. If the operating system is capable of displaying Unicode text in a terminal both ordinary and wide encodings are Unicode encodings such as UTF-8 and UTF-16, respectively. [*Note*: this is the case for Windows-based and many POSIX-based operating systems. — end note] Otherwise, the encodings are implementation-defined.

with

For the purposes of width computation the string is assumed to be in a locale-independent implementation-defined encoding. [*Note*: the implementations are encouraged to use Unicode on platforms capable of displaying Unicode text in a terminal which is the case for Windows-based and many POSIX-based operating systems. — end note]

- Remove the "Optional (possibly in C++23)" section.
- Add poll results.

§ 3. SG16 poll results

Do we want to address this problem in C++20

SF	F	N	A	SA
10	1	2	0	0

Width / precision calculation should be computed in terms of display width?

SF	F	N	A	SA
9	3	1	0	0

Resolve US228 with P1868R0 modified to specify the encoding used to interpret the input text for the purposes of width and precision computation be implementation defined, but not locale sensitive, with non-normative guidance to use Unicode if possible?

SF	F	N	A	SA
7	4	1	0	0

§ 4. LEWG poll results

Forward D1868R1 to LWG for C++20.

SF	F	N	A	SA
5	8	1	0	0

§ 5. Motivating example

To the best of our knowledge, the main use case for the string width and precision format specifiers is to align text when displayed in a terminal with a monospaced font. The motivating example is a columnar view in a typical command-line interface:

```
viz-mbp:test viz$ ls
Die Allgemeine Erklärung der Menschenrechte
Universal Declaration of Human Rights
世界人权宣言
『世界人権宣言』
Всеобщая декларация прав человека
ΟΙΚΟΥΜΕΝΙΚΗ ΔΙΑΚΗΡΥΞΗ ΓΙΑ ΤΑ ΑΝΘΡΩΠΙΝΑ ΔΙΚΑΙΩΜΑΤΑ
```

We would like to be able to produce similar or better output with the C++20 formatting facility using the most natural API, namely dynamic width:

```
// Prints names in num_cols columns of col_width width each.
void print_columns(const std::vector<std::string>& names,
                  int num_cols, int col_width) {
    for (size_t i = 0, size = names.size(); i < size; ++i) {
        std::cout << std::format("{0:{1}}", names[i], col_width);
        if (i % num_cols == num_cols - 1 || i == size - 1) std::cout << '\n';
    }
}

std::vector<std::string> names = {
    "Die Allgemeine Erklärung der Menschenrechte",
    "『世界人権宣言』",
    "Universal Declaration of Human Rights",
    "Всеобщая декларация прав человека",
    "世界人权宣言",
    "ΟΙΚΟΥΜΕΝΙΚΗ ΔΙΑΚΗΡΥΞΗ ΓΙΑ ΤΑ ΑΝΘΡΩΠΙΝΑ ΔΙΚΑΙΩΜΑΤΑ"
};

print_columns(names, 2, 60);
```

Desired output:

Die Allgemeine Erklärung der Menschenrechte	『世界人権宣言』
Universal Declaration of Human Rights	Всеобщая декларация прав человека
世界人权宣言	ΟΙΚΟΥΜΕΝΙΚΗ ΔΙΑΚΗΡΥΞΗ ΓΙΑ ΤΑ ΑΝΘΡΩΠΙΝΑ ΔΙΚΑΙΩΜΑΤΑ

(Note that spacing in front of '『' is part of the character and it is aligned correctly both in the code and in the output.)

§ 6. Prior art

Display width is a well-established concept. In particular, POSIX defines the `wcswidth` function ([[WCWIDTH](#)]) that has the required semantics:

The `wcswidth()` function shall determine the number of column positions required for `n` wide-character codes (or fewer than `n` wide-character codes if a null wide-character code is encountered before `n` wide-character codes are exhausted) in the string pointed to by `pwcs`.

Many languages have implementations of `wcswidth` or similar functionality. Here is an incomplete list of them:

- C `wcwidth` ([MGK25])
- Go `go-runewidth` ([RUNEWIDTH])
- JavaScript `wcwidth.js` ([WCWIDTH-JS])
- Julia `Base.UTF8proc.charwidth`, `Base.strwidth` ([CHARWIDTH]) - a part of the standard library
- Perl `Text::CharWidth` ([TCW])
- Python `wcwidth` ([WCWIDTH-PY]) - used by over 60,000 projects according to the GitHub dependency graph
- Ruby `unicode-display_width` ([UDW]) - used by over 170,000 projects according to the GitHub dependency graph

GitHub code search returns over 500,000 results for "wcwidth" and 180,000 results for "wcswidth".

The number of implementations of this facility together with large usage numbers indicate that it is an important use case. All of the above implementations work exclusively with Unicode.

§ 7. Locale and execution encodings

One of the major design features of the C++20 formatting facility ([P0645]) is locale independence by default with locale-aware formatting available as an opt-in via separate format specifiers. This has an important safety property that the result of `formatted_size` by default does not depend on the global locale and a buffer allocated with this size can be passed safely to `format_to` even if the locale has been changed in the meantime, possibly from another thread. It is desirable to preserve this property for strings for both safety and consistency reasons.

Another observation is that the terminal's encoding is independent from the execution encoding. For example, on Windows it's possible to change the console's code page with `chcp` and `SetConsoleOutputCP` ([SCOCP]) independently of the active code page or the global locale. It is also possible to write Unicode text to a console with `WriteConsoleW` regardless of both the active code page and the console code page. On macOS and Linux, the terminal's encoding is determined by the settings of the terminal emulator application and normally defaults to UTF-8. Changing the encoding is possible but has severe limitations. For example, if you change the terminal encoding from UTF-8 to KOI8-R on macOS `ls` can no longer display even Cyrillic paths even though both UTF-8 and KOI8-R support Cyrillic. Here's an example output with the terminal encoding set to KOI8-R:

consistent with the Windows guidelines and use Unicode by default on this platform. Additionally it should provide an opt-in mechanism to communicate with legacy applications.

§ 9. Precision

Precision, when applied to a string argument, specifies how many characters will be used from the string. It can be used to truncate long strings in the columnar output as in the motivating example shown earlier. Because it works with a single argument and only for some argument types it is not particularly useful for truncating output to satisfy storage requirements. `format_to_n` should be used for the latter instead. The semantics of floating-point precision is also unrelated to storage.

Since precision and width address the same use case, we think that they should be measured in the same units.

§ 10. Proposal

To address the main use case, we propose using the display width of a string, i.e. the number of column positions needed to display the string in a terminal, for both width and precision.

There is a spectrum of solutions to the problem of estimating display width, from always wrong (return 42 times the number of code units) and almost always wrong (code units and `printf`) to always correct (model the terminal's logic of width computation). We would like to take a pragmatic approach leaning towards the correct side of the spectrum but without introducing too much complexity. This can be accomplished by defining ranges of characters that are guaranteed to be handled correctly on a capable terminal with an option of refining the definition as technology matures and Unicode handling bugs observed today are fixed. With our approach a program can produce high-quality output which is always correct by escaping characters for which width computation is not supported and is readable in many common cases, greatly improving on `printf`.

To satisfy the locale-independence property we propose that for the purposes of display width computation the default should be Unicode on systems that support display of Unicode text in a terminal or fixed implementation-defined encodings otherwise. In particular this allows using EBCDIC on z/OS and ASCII on resource-constrained embedded systems that may not want to provide even minimal Unicode handling capabilities. On Unicode-capable systems both `char` and `wchar_t` strings should use Unicode encodings (e.g. UTF-8 and UTF-16 respectively) by default. This will enable portable code with optional transcoding at the system API boundaries (see [\[P1238\]](#)) and seamless integration with APIs that support Unicode such as `WriteConsoleW` on Windows without data loss.

Using a fixed system encoding is completely safe because formatting functions don't do any transcoding. So the worst thing that can happen is that the display width will be estimated incorrectly leading to misaligned text which is what already happens when you pass a variable-width string to `printf`. This is also not novel, for example `std::filesystem` also acknowledges the existence of system dependent encodings:

The native encoding of an ordinary character string is the operating system dependent current encoding for pathnames.

For Unicode, the first step in computing width is to break the string into grapheme clusters because the latter correspond to user-perceived characters ([UAX29]). Then the width should be adjusted to account for graphemes that take two column positions as it is done, for example, in the Unicode implementation of `wcswidth` by Markus Kuhn ([MGK25]). Non-printable characters such as control characters do not contribute to width and it should be a user's responsibility to ensure that the input string does not contain such characters as well as leading combining characters and modifier letters that may compose after concatenation.

Width estimation can be done efficiently with a single pass over the input and optimized for the case of no variable-width characters. It has zero overhead when no width is specified or when formatting non-string arguments.

We also propose adding a new format specifier in C++23 for computing display width of a string argument based on the locale's encoding, for example:

```
std::locale::global(std::locale("ru_RU.KOI8-R"));
std::string message = std::format("{:6ls}", "\xd4\xc5\xd3\xd4"); // "тест"
// message == "\xd4\xc5\xd3\xd4" ("тест" in KOI8-R)
```

This will support display width estimation for ordinary and wide execution encodings. We think that the current proposal is in line with SG16: Unicode Direction ([P1238]) goal of "Designing for where we want to be and how to get there" because it creates a clear path for the future `charN_t` overloads of `std::format` to have the desired behavior and be consistent with the C++20 formatting facility which currently supports `char` and `wchar_t`.

§ 11. Why not code units?

It might seem tempting at first to measure width in code units because it is simple and avoids the encoding question. However, it is not very useful in addressing practical use cases. Also it is an

evolutionary deadend because standardizing code units for `char` and `wchar_t` overloads by default would create an incentive for doing the same in `charN_t` overloads or introduce a confusing difference in behavior. One might argue that if we do the latter it may push users to the `charN_t` overloads but intentionally designing an inferior API and creating inconvenience for users for the goal that may never realise seems wrong. Measuring width in code units in the `fmt` library was surprising to some users resulting in bug reports and eventually switching to higher-level units.

Code units are even less adequate for precision, because they can result in invalid output. For example

```
std::string s = std::format("{:.2}", "\x41\xCC\x81");
```

would result in `s` containing `"\x41\xCC"` if precision was measured in code units which is clearly broken. In Python's `str.format` precision is measured in code points which prevents this issue.

`printf`, which works with code units, can only handle basic Latin in UTF-8, so even formatting of common English words containing accents is problematic. For example:

```
printf("%10s - %s\n", "bistro", "a small or unpretentious restaurant");  
printf("%10s - %s\n", "café",  
       "a usually small and informal establishment serving various refreshments");
```

prints

```
bistro - a small or unpretentious restaurant  
café - a usually small and informal establishment serving various refreshments
```

or

```
bistro - a small or unpretentious restaurant  
café - a usually small and informal establishment serving various refreshments
```

depending on how `é` is represented.

If we want to truncate the output

```
printf("%.4s...\n", "bistro");  
printf("%.4s...\n", "café");
```

the result is even worse:

```
bist...  
caf<C3>...
```

§ 12. Limitations

Unlike terminals, GUI editors often use proportional fonts or fonts that claim to be monospaced but treat some characters such that their width is not an integer multiple of the other. Therefore width, regardless of how it is defined, is inherently limited there. However, it can still be useful if the input domain is restricted. Possible use cases are aligning numbers, text in ASCII or other subset of Unicode, or adding code indentation:

```
// Prints text prefixed with indent spaces.  
void print_indented(int indent, std::string_view text) {  
    std::cout << fmt::format("{0:>{1}}{2}\n", "", indent, text);  
}
```

Our definition of width fully support these use cases and gives better results than `printf` for Unicode subranges.

§ 13. Examples

```
#include <format>
#include <iostream>
#include <stdio.h>

struct input {
    const char* text;
    const char* info;
};

int main() {
    input inputs[] = {
        {"Text", "Description"},
        {"-----",
         "-----"},
        {"\x41", "U+0041 { LATIN CAPITAL LETTER A }"},
        {"\xC3\x81", "U+00C1 { LATIN CAPITAL LETTER A WITH ACUTE }"},
        {"\x41\xCC\x81",
         "U+0041 U+0301 { LATIN CAPITAL LETTER A } { COMBINING ACUTE ACCENT }"},
        {"\xc4\xb2", "U+0132 { LATIN CAPITAL LIGATURE IJ }"}, // IJ
        {"\xce\x94", "U+0394 { GREEK CAPITAL LETTER DELTA }"}, // Δ
        {"\xd0\xa9", "U+0429 { CYRILLIC CAPITAL LETTER SHCHA }"}, // Ш
        {"\xd7\x90", "U+05D0 { HEBREW LETTER ALEF }"}, // א
        {"\xd8\xb4", "U+0634 { ARABIC LETTER SHEEN }"}, // ش
        {"\xe3\x80\x89", "U+3009 { RIGHT-POINTING ANGLE BRACKET }"}, // >
        {"\xe7\x95\x8c", "U+754C { CJK Unified Ideograph-754C }"}, // 界
        {"\xf0\x9f\xa6\x84", "U+1F921 { UNICORN FACE }"}, // 🦄
        {"\xf0\x9f\x91\xa8\xe2\x80\x8d\xf0\x9f\x91\xa9\xe2\x80\x8d"
         "\xf0\x9f\x91\xa7\xe2\x80\x8d\xf0\x9f\x91\xa6",
         "U+1F468 U+200D U+1F469 U+200D U+1F467 U+200D U+1F466 "
         "{ Family: Man, Woman, Girl, Boy } " } // 👨👩👧👦
    };

    std::cout << "\nstd::format with the current proposal:\n";
    for (auto input: inputs) {
        std::cout << std::format("{:>5} | {} \n", input.text, input.info);
    }

    std::cout << "\nprintf:\n";
    for (auto input: inputs) {
```

```

    printf("%5s | %s\n", input.text, input.info);
}
}

```

Output on macOS Terminal:

```

std::format with the current proposal:
Text | Description
-----|-----
A | U+0041 { LATIN CAPITAL LETTER A }
Á | U+00C1 { LATIN CAPITAL LETTER A WITH ACUTE }
Ã | U+0041 U+0301 { LATIN CAPITAL LETTER A } { COMBINING ACUTE ACCENT }
IJ | U+0132 { LATIN CAPITAL LIGATURE IJ }
Δ | U+0394 { GREEK CAPITAL LETTER DELTA }
Ш | U+0429 { CYRILLIC CAPITAL LETTER SHCHA }
א | U+05D0 { HEBREW LETTER ALEF }
ﺀ | U+0634 { ARABIC LETTER SHEEN }
> | U+3009 { RIGHT-POINTING ANGLE BRACKET }
界 | U+754C { CJK Unified Ideograph-754C }
🦄 | U+1F921 { UNICORN FACE }
👨👩👧👦 | U+1F468 U+200D U+1F469 U+200D U+1F467 U+200D U+1F466 { Family: Man, Woman, Girl, Boy }

printf:
Text | Description
-----|-----
A | U+0041 { LATIN CAPITAL LETTER A }
Á | U+00C1 { LATIN CAPITAL LETTER A WITH ACUTE }
Ã | U+0041 U+0301 { LATIN CAPITAL LETTER A } { COMBINING ACUTE ACCENT }
IJ | U+0132 { LATIN CAPITAL LIGATURE IJ }
Δ | U+0394 { GREEK CAPITAL LETTER DELTA }
Ш | U+0429 { CYRILLIC CAPITAL LETTER SHCHA }
א | U+05D0 { HEBREW LETTER ALEF }
ﺀ | U+0634 { ARABIC LETTER SHEEN }
> | U+3009 { RIGHT-POINTING ANGLE BRACKET }
界 | U+754C { CJK Unified Ideograph-754C }
🦄 | U+1F921 { UNICORN FACE }
👨👩👧👦 | U+1F468 U+200D U+1F469 U+200D U+1F467 U+200D U+1F466 { Family: Man, Woman, Girl, Boy }

```

Notice that the `printf` output is completely misaligned except for one case because width is measured in code units.

Output on Windows with console codepage set to 65001 (UTF-8) and the active code page unchanged:

```

std::format with the current proposal:
Text | Description
-----
A | U+0041 { LATIN CAPITAL LETTER A }
Á | U+00C1 { LATIN CAPITAL LETTER A WITH ACUTE }
A´ | U+0041 U+0301 { LATIN CAPITAL LETTER A } { COMBINING ACUTE ACCENT }
IJ | U+0132 { LATIN CAPITAL LIGATURE IJ }
Δ | U+0394 { GREEK CAPITAL LETTER DELTA }
Ш | U+0429 { CYRILLIC CAPITAL LETTER SHCHA }
א | U+05D0 { HEBREW LETTER ALEF }
ﺀ | U+0634 { ARABIC LETTER SHEEN }
☞ | U+3009 { RIGHT-POINTING ANGLE BRACKET }
𐤀 | U+754C { CJK Unified Ideograph-754C }
🦄 | U+1F921 { UNICORN FACE }
👨 👩 👧 👦 | U+1F468 U+200D U+1F469 U+200D U+1F467 U+200D U+1F466 { Family: Man, Woman, Girl, Boy }

printf:
Text | Description
-----
A | U+0041 { LATIN CAPITAL LETTER A }
Á | U+00C1 { LATIN CAPITAL LETTER A WITH ACUTE }
A´ | U+0041 U+0301 { LATIN CAPITAL LETTER A } { COMBINING ACUTE ACCENT }
IJ | U+0132 { LATIN CAPITAL LIGATURE IJ }
Δ | U+0394 { GREEK CAPITAL LETTER DELTA }
Ш | U+0429 { CYRILLIC CAPITAL LETTER SHCHA }
א | U+05D0 { HEBREW LETTER ALEF }
ﺀ | U+0634 { ARABIC LETTER SHEEN }
☞ | U+3009 { RIGHT-POINTING ANGLE BRACKET }
𐤀 | U+754C { CJK Unified Ideograph-754C }
🦄 | U+1F921 { UNICORN FACE }
👨 👩 👧 👦 | U+1F468 U+200D U+1F469 U+200D U+1F467 U+200D U+1F466 { Family: Man, Woman, Girl, Boy }

```

The Windows console doesn't handle combining accents and emoji correctly which is unrelated to the question of width. Although it is possible to implement a workaround for this platform we advise against it. If the output is incorrect it is reasonable to expect the alignment to be incorrect as well. The new Windows Terminal reportedly handles emoji correctly. Console bugs aside, `printf` has the same issues on Windows as on macOS.

Notice that although the Windows console is unable to display CJK Unified Ideograph-754C, the width is still computed correctly and a placeholder character is displayed instead. This is a very nice example of a fallback behavior, in this case done by the terminal itself.

Output on GNOME Terminal 3.32.1 in Linux:

```

std::format with the current proposal:
Text | Description
-----|-----
A | U+0041 { LATIN CAPITAL LETTER A }
Á | U+00C1 { LATIN CAPITAL LETTER A WITH ACUTE }
Á | U+0041 U+0301 { LATIN CAPITAL LETTER A } { COMBINING ACUTE ACCENT }
IJ | U+0132 { LATIN CAPITAL LIGATURE IJ }
Δ | U+0394 { GREEK CAPITAL LETTER DELTA }
Ш | U+0429 { CYRILLIC CAPITAL LETTER SHCHA }
א | U+05D0 { HEBREW LETTER ALEF }
ش | U+0634 { ARABIC LETTER SHEEN }
> | U+3009 { RIGHT-POINTING ANGLE BRACKET }
界 | U+754C { CJK Unified Ideograph-754C }
🦄 | U+1F921 { UNICORN FACE }
👨👩👧👦 | U+1F468 U+200D U+1F469 U+200D U+1F467 U+200D U+1F466 { Family: Man, Woman, Girl, Boy }

printf:
Text | Description
-----|-----
A | U+0041 { LATIN CAPITAL LETTER A }
Á | U+00C1 { LATIN CAPITAL LETTER A WITH ACUTE }
Á | U+0041 U+0301 { LATIN CAPITAL LETTER A } { COMBINING ACUTE ACCENT }
IJ | U+0132 { LATIN CAPITAL LIGATURE IJ }
Δ | U+0394 { GREEK CAPITAL LETTER DELTA }
Ш | U+0429 { CYRILLIC CAPITAL LETTER SHCHA }
א | U+05D0 { HEBREW LETTER ALEF }
ش | U+0634 { ARABIC LETTER SHEEN }
> | U+3009 { RIGHT-POINTING ANGLE BRACKET }
界 | U+754C { CJK Unified Ideograph-754C }
🦄 | U+1F921 { UNICORN FACE }
👨👩👧👦 | U+1F468 U+200D U+1F469 U+200D U+1F467 U+200D U+1F466 { Family: Man, Woman, Girl, Boy }

```

Output on Konsole 18.12.3 in Linux:

```

std::format with the current proposal:
Text | Description
-----|-----
A | U+0041 { LATIN CAPITAL LETTER A }
Á | U+00C1 { LATIN CAPITAL LETTER A WITH ACUTE }
Á | U+0041 U+0301 { LATIN CAPITAL LETTER A } { COMBINING ACUTE ACCENT }
IJ | U+0132 { LATIN CAPITAL LIGATURE IJ }
Δ | U+0394 { GREEK CAPITAL LETTER DELTA }
Ш | U+0429 { CYRILLIC CAPITAL LETTER SHCHA }
א | U+05D0 { HEBREW LETTER ALEF }
ش | U+0634 { ARABIC LETTER SHEEN }
> | U+3009 { RIGHT-POINTING ANGLE BRACKET }
界 | U+754C { CJK Unified Ideograph-754C }
🦄 | U+1F921 { UNICORN FACE }
👨👩👧👦 | U+1F468 U+200D U+1F469 U+200D U+1F467 U+200D U+1F466 { Family: Man, Woman, Girl, Boy }

printf:
Text | Description
-----|-----
A | U+0041 { LATIN CAPITAL LETTER A }
Á | U+00C1 { LATIN CAPITAL LETTER A WITH ACUTE }
Á | U+0041 U+0301 { LATIN CAPITAL LETTER A } { COMBINING ACUTE ACCENT }
IJ | U+0132 { LATIN CAPITAL LIGATURE IJ }
Δ | U+0394 { GREEK CAPITAL LETTER DELTA }
Ш | U+0429 { CYRILLIC CAPITAL LETTER SHCHA }
א | U+05D0 { HEBREW LETTER ALEF }
ش | U+0634 { ARABIC LETTER SHEEN }
> | U+3009 { RIGHT-POINTING ANGLE BRACKET }
界 | U+754C { CJK Unified Ideograph-754C }
🦄 | U+1F921 { UNICORN FACE }
👨👩👧👦 | U+1F468 U+200D U+1F469 U+200D U+1F467 U+200D U+1F466 { Family: Man, Woman, Girl, Boy }

```


GNOME Terminal and Konsole which are two major terminal emulators on Linux also cannot handle complex emoji yet but otherwise produce similar results.

§ 14. Implementation

The proposal is implemented in the `fmt` library, successfully tested on a variety of platforms, and will become the default for both `char` and `wchar_t` strings since the results showed great improvement in width estimation compared to using code units and code points. The implementation is very simple and required only integration of an existing grapheme cluster break and `wcswidth` implementations which was done in less than a day.

We tested our implementation on macOS Terminal version 2.9.5 (421.2), Windows console on Windows version 10.0.17763.737 and GNOME Terminal 3.32.1 on Linux verifying that the display width is consistent for at least the following Unicode blocks according to our definition of width and produces visually aligned results:

Block range	Block name
=====	=====
U+0000..U+007F	Basic Latin
U+0080..U+00FF	Latin-1 Supplement
U+0100..U+017F	Latin Extended-A
U+0180..U+024F	Latin Extended-B
U+0250..U+02AF	IPA Extensions
U+02B0..U+02FF	Spacing Modifier Letters
U+0300..U+036F	Combining Diacritical Marks
U+0370..U+03FF	Greek and Coptic
U+0400..U+04FF	Cyrillic
U+0500..U+052F	Cyrillic Supplement
U+0530..U+058F	Armenian
U+0590..U+05FF	Hebrew
U+0600..U+06FF	Arabic
U+0700..U+074F	Syriac
U+0750..U+077F	Arabic Supplement
U+0780..U+07BF	Thaana
U+07C0..U+07FF	NKo
U+0800..U+083F	Samaritan
U+0840..U+085F	Mandaic
U+0860..U+086F	Syriac Supplement
U+08A0..U+08FF	Arabic Extended-A
U+0900..U+097F	Devanagari
U+0980..U+09FF	Bengali
U+0A00..U+0A7F	Gurmukhi

Even this small subset of Unicode includes 5 out of top 10 writing scripts ranked by active usage ([\[SCRIPTS\]](#)) with billions of active users. For comparison, width in `printf` cannot even handle a single writing script. More importantly, our approach permits refining the definition in the future if there is interest in doing so. It will mostly require researching the status of Unicode support on terminals and minimal or no changes to the implementation.

Additionally we looked at the Unicode block U+1F300..U+1F5FF Miscellaneous Symbols and Pictographs. Support for code points in this block varies, for example, in the Windows console none of them were displayed correctly, with or without width. In macOS Terminal most of the symbols had width 2 with a few exceptions. Therefore we recommend not declaring any support for this block and using the default of 2. This will have reasonable behavior on systems that support emoji and make it clear that these symbols may need escaping or other fallback mechanism.

§ 15. Wording

Modify [\[format.string.std\]/p7](#) as follows:

The positive-integer in width is a decimal integer defining the minimum field width. If width is not specified, there is no minimum field width, and the field width is determined based on the content of the field.

The width of a string is defined as the estimated number of column positions appropriate for displaying it in a terminal. [Note: This is similar to the semantics of the POSIX `wcswidth` function. — end note]

For the purposes of width computation a string is assumed to be in a locale-independent implementation-defined encoding. Implementations should use a Unicode encoding on platforms capable of displaying Unicode text in a terminal. [Note: This is the case for Windows-based and many POSIX-based operating systems. — end note]

For a string in a Unicode encoding, implementations should estimate the width of a string as the sum of estimated widths of the first code points in its extended grapheme clusters as defined by Unicode® Standard Annex #29 Unicode Text Segmentation. The estimated width of the following code points is 2:

U+1100 .. U+115F
U+2329
U+232A
U+2E80 .. U+303E
U+3040 .. U+A4CF
U+AC00 .. U+D7A3
U+F900 .. U+FAFF
U+FE10 .. U+FE19
U+FE30 .. U+FE6F
U+FF00 .. U+FF60
U+FFE0 .. U+FFE6
U+20000 .. U+2FFFD
U+30000 .. U+3FFFD
U+1F300 .. U+1F64F
U+1F900 .. U+1F9FF

The estimated width of other code points is 1.

For a string in a non-Unicode encoding, the width of a string is unspecified.

Modify [\[format.string.std\]/p9](#) as follows:

The nonnegative-integer in precision is a decimal integer defining the precision or maximum field size. It can only be used with floating-point and string types. For floating-point types this field specifies the formatting precision. ~~For string types it specifies how many characters will be used from the string.~~ For string types, this field provides an upper bound for the estimated width of the prefix of the input string that is copied into the output. For a string in a Unicode encoding, the formatter copies to the output the longest prefix of whole extended grapheme clusters whose estimated width is no greater than the precision.

Add to Normative references [\[intro.refs\]](#):

— Unicode® 12.1.0 Standard Annex #29 Unicode Text Segmentation

§ 16. Acknowledgements

We would like to thank Tom Honermann for bringing the issue of ambiguous width to our attention and Henri Sivonen for a very detailed and insightful post on various definitions of Unicode string length ([\[LENGTH\]](#)) which helped us in researching the topic.

§ References

§ Informative References

[CHARWIDTH]

[Julia Documentation, Strings, Base.UTF8proc.charwidth](#). URL:
<https://docs.julialang.org/en/v0.6/stdlib/strings/#Base.UTF8proc.charwidth>

[LENGTH]

Henri Sivonen. [It's Not Wrong that "👨".length == 7 But It's Better that "👨".len\(\) == 17 and Rather Useless that len\("👨"\) == 5](#). URL: <https://hsivonen.fi/string-length/>

[LWG3290]

[LWG Issue 3290: Are std::format field widths code units, code points, or something else?](#). URL:
<https://cplusplus.github.io/LWG/issue3290>

[MGK25]

Markus Kuhn. [An implementation of `wcwidth\(\)` and `wcswidth\(\)` for Unicode](https://www.cl.cam.ac.uk/~mgk25/ucs/wcwidth.c). URL: <https://www.cl.cam.ac.uk/~mgk25/ucs/wcwidth.c>

[P0645]

Victor Zverovich. [Text Formatting](https://wg21.link/p0645). URL: <https://wg21.link/p0645>

[P1238]

Tom Honermann; et al. [SG16: Unicode Direction](https://wg21.link/p1238). URL: <https://wg21.link/p1238>

[RUNEWIDTH]

[wcwidth for golang](https://github.com/mattn/go-runewidth). URL: <https://github.com/mattn/go-runewidth>

[SCOCP]

[Console Reference, SetConsoleOutputCP function](https://docs.microsoft.com/en-us/windows/console/setconsoleoutputcp). URL: <https://docs.microsoft.com/en-us/windows/console/setconsoleoutputcp>

[SCRIPTS]

[List of writing systems, List of writing scripts by adoption](https://en.wikipedia.org/wiki/List_of_writing_systems#List_of_writing_scripts_by_adoption). URL: https://en.wikipedia.org/wiki/List_of_writing_systems#List_of_writing_scripts_by_adoption

[TCW]

[Text::CharWidth - Get number of occupied columns of a string on terminal](https://metacpan.org/pod/Text::CharWidth). URL: <https://metacpan.org/pod/Text::CharWidth>

[UAX29]

[Unicode® Standard Annex #29: Unicode Text Segmentation](https://unicode.org/reports/tr29/). URL: <https://unicode.org/reports/tr29/>

[UDW]

[unicode-display_width: Monospace Unicode character width in Ruby](https://github.com/janlelis/unicode-display_width) . URL: https://github.com/janlelis/unicode-display_width

[WCSWIDTH]

[`wcswidth\(\)`, The Open Group Base Specifications Issue 6 IEEE Std 1003.1, 2004 Edition](https://pubs.opengroup.org/onlinepubs/009696799/functions/wcswidth.html). URL: <https://pubs.opengroup.org/onlinepubs/009696799/functions/wcswidth.html>

[WCWIDTH-JS]

[wcwidth.js: a javascript porting of C's wcwidth\(\)](https://github.com/mycoboco/wcwidth.js). URL: <https://github.com/mycoboco/wcwidth.js>

[WCWIDTH-PY]

[wcwidth: Python library that measures the width of unicode strings rendered to a terminal](https://github.com/jquast/wcwidth). URL: <https://github.com/jquast/wcwidth>

[WINI18N]

[Windows documentation, Internationalization for Windows Applications, Code Pages](https://docs.microsoft.com/en-us/windows/win32/intl/code-pages). URL: <https://docs.microsoft.com/en-us/windows/win32/intl/code-pages>